

Ziel der Aufgabe

Erstelle ein **fokussiertes, praxistaugliches Konzept (max. 6 Seiten)** für den Aufbau einer **plattformweiten Lösung für asynchrone Kommunikation zwischen Services** – inklusive **kleinem Terraform-Prototyp**.

Der Fokus liegt auf:

- fundierten Architekturentscheidungen in einer **AWS-Umgebung**
- klaren Trade-offs und pragmatischen Entscheidungen
- **konkreter Umsetzbarkeit (MVP + Prototyp)**
- Developer Experience und Adoption
- stabilem, sicherem Betrieb

Wichtig: Priorisiere bewusst und fokussiere dich auf die wichtigsten Teile.

Ausgangslage

Du arbeitest als Platform Engineer in einem Plattform-Team (1 Manager, 4 Engineers).

Das Produkt ist eine AWS-basierte SaaS-Lösung mit aktuell 12 Productteams mit 20+ Services.

Aktuelle Situation:

- Unterschiedliche Ansätze für asynchrone Kommunikation (Queues, Cron, eigene Worker)
- Kein Standard für Events, Retries, Fehlerhandling oder Monitoring
- Unklare Ownership von Messages / Events
- Operative Probleme (verlorene Messages, Debugging schwierig)
- Kein klarer Golden Path

Ziel ist eine **einheitliche, entwicklerfreundliche Plattform-Fähigkeit für asynchrone Kommunikation**.

Aufgabe

Erstelle ein Konzept und einen kleinen Prototyp für eine Async-Communication Plattform.

Struktur und Schwerpunktsetzung wählst du selbst.

Deine Lösung sollte jedoch erkennbar folgende Aspekte adressieren:

1. Architektur & Design

- Auswahl geeigneter AWS Services
- Event- vs. Command-basierte Kommunikation
- Event Schema / Envelope
- Fehlerhandling (Retries, DLQ, Idempotenz)
- Zugang, Security & Isolation

Triff klare Entscheidungen und erkläre bewusst, was du **nicht** baust.

2. Terraform Prototyp (Hands-on Teil)

Erstelle einen **kleinen, fokussierten Terraform-Skeleton**.

Fokus:

- Struktur und Verständlichkeit
- sinnvolle Defaults
- Wiederverwendbarkeit

Kein vollständiges Produktions-Setup oder Modul-System notwendig.

3. Developer Experience & Golden Path

- Wie nutzen Entwickler die Plattform konkret?
- Wie sieht ein „Happy Path“ aus?
- Welche Abstraktionen (z. B. SDK, Templates, APIs) bietest du an?

4. Betrieb & Stabilität

- Monitoring & Observability
- Debugging von Event-Flows
- Umgang mit Fehlern und Incidents

5. Vision & Weiterentwicklung (optional, kurz)

- Wie entwickelt sich diese Capability über die Zeit weiter?

Fokus auf wichtigste nächste Schritte, kein Overengineering